



CREATING TECHNOLOGY FOR **MUSIC**

SUPERCOLLIDER

for the Creative Musician

A Practical Guide

Eli Fieldsteel

SUPERCOLLIDER FOR THE CREATIVE MUSICIAN

Creating Technology for Music

Series Editor

V. J. Manzo, Associate Professor of Music Technology and
Cognition at Worcester Polytechnic Institute

SuperCollider for the Creative Musician: A Practical Guide

Eli Fieldsteel

SUPERCOLLIDER FOR THE CREATIVE MUSICIAN

A Practical Guide

Eli Fieldsteel

OXFORD
UNIVERSITY PRESS

OXFORD
UNIVERSITY PRESS

Oxford University Press is a department of the University of Oxford. It furthers the University's objective of excellence in research, scholarship, and education by publishing worldwide. Oxford is a registered trade mark of Oxford University Press in the UK and certain other countries.

Published in the United States of America by Oxford University Press
198 Madison Avenue, New York, NY 10016, United States of America.

© Oxford University Press 2024

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, without the prior permission in writing of Oxford University Press, or as expressly permitted by law, by license, or under terms agreed with the appropriate reproduction rights organization. Inquiries concerning reproduction outside the scope of the above should be sent to the Rights Department, Oxford University Press, at the address above.

You must not circulate this work in any other form
and you must impose this same condition on any acquirer.

Library of Congress Cataloging-in-Publication Data

Names: Fieldsteel, Eli, author.

Title: SuperCollider for the creative musician : a practical guide / Eli Fieldsteel.

Description: New York, NY : Oxford University Press, 2024. |

Series: Creating technology for music |

Includes bibliographical references and index.

Identifiers: LCCN 2023032483 (print) | LCCN 2023032484 (ebook) |

ISBN 9780197616994 (hardback) | ISBN 9780197617007 (paperback) |

ISBN 9780197617021 (epub) | ISBN 9780197617014 (updf) |

ISBN 9780197617038 (digital-online) | ISBN 9780197617045 (digital-online)

Subjects: LCSH: SuperCollider (Computer file) |

Computer composition (Music) | Software synthesizers.

Classification: LCC ML74.4.S86 F54 2024 (print) |

LCC ML74.4.S86 (ebook) | DDC 781.3/4536—dc23/eng/230718

LC record available at <https://lcn.loc.gov/2023032483>

LC ebook record available at <https://lcn.loc.gov/2023032484>

DOI: 10.1093/oso/9780197616994.001.0001

Paperback printed by Sheridan Books, Inc., United States of America

Hardback printed by Bridgeport National Bindery, Inc., United States of America

To Henry, my cat, who passed away shortly after I completed the first manuscript draft. He spent his last few weeks curled up in my lap listening to bleeps and bloops. He was an invaluable collaborator.

CONTENTS

ACKNOWLEDGMENTS [XI](#)

ABOUT THE COMPANION WEBSITE [XIII](#)

INTRODUCTION [XV](#)

PART I FUNDAMENTALS 1

CHAPTER 1 CORE PROGRAMMING CONCEPTS 3

- 1.1 Overview [3](#)
- 1.2 A Tour of the Environment [3](#)
- 1.3 An Object-Oriented View of the World [5](#)
- 1.4 Writing, Understanding, and Evaluating Code [7](#)
- 1.5 Getting Help [13](#)
- 1.6 A Tour of Classes and Methods [17](#)
- 1.7 Randomness [27](#)
- 1.8 Conditional Logic [29](#)
- 1.9 Iteration [32](#)
- 1.10 Summary [34](#)

CHAPTER 2 ESSENTIALS OF MAKING SOUND 35

- 2.1 Overview [35](#)
- 2.2 Booting the Audio Server [35](#)
- 2.3 Unit Generators [37](#)
- 2.4 UGen Functions [42](#)
- 2.5 Envelopes [51](#)
- 2.6 Multichannel Signals [59](#)
- 2.7 SynthDef and Synth [65](#)
- 2.8 Alternate Expression of Frequency and Amplitude [71](#)
- 2.9 Helpful Server Tools [72](#)

PART II CREATIVE TECHNIQUES 77**CHAPTER 3 SYNTHESIS 79**

- 3.1 Overview 79
- 3.2 Additive Synthesis 79
- 3.3 Modulation Synthesis 84
- 3.4 Wavetable Synthesis 93
- 3.5 Filters and Subtractive Synthesis 98
- 3.6 Modal Synthesis 106
- 3.7 Waveform Distortion 109
- 3.8 Conclusions and Further Ideas 113

CHAPTER 4 SAMPLING 115

- 4.1 Overview 115
- 4.2 Buffers 115
- 4.3 Sampling UGens 122
- 4.4 Recording UGens 129
- 4.5 Granular Synthesis 139

CHAPTER 5 SEQUENCING 146

- 5.1 Overview 146
- 5.2 Routines and Clocks 146
- 5.3 Patterns 156
- 5.4 Additional Techniques for Pattern Composition 170
- 5.5 Real-Time Pattern Control 176

CHAPTER 6 SIGNAL PROCESSING 183

- 6.1 Overview 183
- 6.2 Signal Flow Concepts on the Audio Server 183
- 6.3 Delay-Based Processing 198
- 6.4 Real-Time Granular Synthesis 204

CHAPTER 7 EXTERNAL CONTROL 211

- 7.1 Overview 211
- 7.2 MIDI 211
- 7.3 OSC 220
- 7.4 Other Options for External Control 227

CHAPTER 8 GRAPHICAL USER INTERFACES 233

- 8.1 Overview 233
- 8.2 Basic GUI Principles 233
- 8.3 Intermediate GUI Techniques 241
- 8.4 Custom Graphics 245

PART III LARGE-SCALE PROJECTS 249**CHAPTER 9 CONSIDERATIONS FOR LARGE-SCALE PROJECTS 251**

- 9.1 Overview 251
- 9.2 `waitForBoot` 252
- 9.3 Asynchronous Commands 253
- 9.4 Initialization and Cleanup Functions 255
- 9.5 The Startup File 260
- 9.6 Working with Multiple Code Files 260

CHAPTER 10 AN EVENT-BASED STRUCTURE 263

- 10.1 Overview 263
- 10.2 Expressing Musical Events Through Code 263
- 10.3 Organizing Musical Events 266
- 10.4 Navigating and Rehearsing an Event-Based Composition 271
- 10.5 Indeterminacy in an Event-Based Composition 273

CHAPTER 11 A STATE-BASED STRUCTURE 278

- 11.1 Overview 278
- 11.2 Simple State Control 279
- 11.3 Composite States 282
- 11.4 Patterns in a State-Based Composition 285
- 11.5 One-Shots in a State-Based Composition 288
- 11.6 Signal Processing in a State-Based Composition 290
- 11.7 Performing a State-Based Composition 293

CHAPTER 12 LIVE CODING 298

- 12.1 Overview 298
- 12.2 A Live Coding Problem and Solution 299
- 12.3 `NodeProxy` 300
- 12.4 Additional `NodeProxy` Features 308
- 12.5 `TaskProxy` 314
- 12.6 Recording a Live Coding Performance 317

INDEX 321

ACKNOWLEDGMENTS

This book is the result of over a decade of using the SuperCollider programming language for music composition and experimental sound art, a journey that would not have been possible without the support of many others. My brother, Nathan, introduced me to SuperCollider in 2007 and provided the initial spark of interest. During my first year of graduate study in music composition, my peers L. Scott Price and Ilya Rostovtsev were creating code-based music that radically expanded the scope of my musical world. In 2010, I took a summer course with James Lipton and Ron Kuivila at Wesleyan University, which put so many missing pieces into place and allowed me to properly begin self-teaching. During my graduate studies, my advisor Russell Pinkston was an incredible teacher and an endless fount of knowledge. The Companion Code materials that accompany this book include audio samples from musicians who have enriched my life through collaborations, performances, and recording sessions, notably flutist Kenzie Slottow, percussionist Adam Groh, and saxophonist Nathan Mandel. A few close friends, colleagues, and former teachers—Jake Metz, Jon C. Nelson, and Stephen Taylor—generously volunteered their time to read and provide feedback on parts of the manuscript as it took shape. I also owe a great deal to the many gurus and developers of the SuperCollider community, who were always kind and helpful in online forums when I had questions. Finally, I am deeply grateful for my wife, Miriam, who is one of the most compassionate and supportive people I have ever met.

Thank you to everyone who helped me become the musician I am today.

ABOUT THE COMPANION WEBSITE

www.oup.com/us/supercolliderforthecreativemusician

We have created a website to accompany *SuperCollider for the Creative Musician: A Practical Guide*. Throughout this book, in-line examples of SuperCollider code appear in chapters, and additionally, each chapter pairs with one or more large-scale SuperCollider code projects. Collectively, these examples serve to amplify and augment concepts discussed in the book. The former (Code Examples) focus on bite-size ideas, while the latter (Companion Codes) explore broader topics in greater detail. All these examples are available on this website. When Companion Code files are referenced in the text, they are accompanied with Oxford's symbol .

The purpose of this website is to foster a hands-on learning approach, by enabling the reader to quickly explore code on their own, while avoiding the burden of having to copy code by hand into a computer. When engaging with a specific chapter, the reader is encouraged to download and open the corresponding Code Examples and Companion Codes in SuperCollider, so that they are readily accessible whenever the urge to experiment arises.

INTRODUCTION

Welcome to *SuperCollider for the Creative Musician: A Practical Guide*. I hope you'll find this book rewarding, ear-opening, and fun.

A little bit about myself: I've been a musician for most of my life. I took piano lessons when I was young, learned other instruments throughout middle school and high school, and became seriously interested in music composition when I was about 15 years old. At the same time, math and computer science were always among my strongest aptitudes. I took a handful of college math courses during my last year of high school, and I had studied the manual for my TI-83 calculator to the point where I was able to start coding semi-complex programs on my own. I dabbled in commercial audio software (GarageBand, mostly) in high school and college, but it wasn't until I'd started graduate studies in music composition that I began to realize the depth to which music and computational thinking could be artfully combined. Discovering SuperCollider and seeing/hearing what it could do opened creative doors that I didn't even know existed. Getting it to do what I wanted, however . . . was a different matter. I'd never had any formal training in computer programming beyond a few spontaneously taken college classes, and as coding platforms go, SuperCollider is quirky, so it was a slow uphill climb for me. After several years, as I finally began feeling competent and self-sufficient, I began taking notes on things I wish I'd known from day one. These notes evolved into a sort of curriculum, which manifested as an ongoing series of video tutorials published on YouTube, and now, this book.

Why use code to make music? Under the hood, all audio software is built with code, but often presented to the user as a rich graphical interface layer meant to provide an intuitive means of control, helping new users start making sounds quickly and easily, without really needing to know much about the underlying principles. It's wonderful that these tools exist, but for users wanting to get their hands deeper into the machine to do things that are more precise or customized, these tools can feel restrictive because the inner workings are "concealed" behind a thick graphical user interface. Programs like SuperCollider, which allow the user to work directly with code, provide far more direct control, power, and flexibility over the types of sounds that happen, when they happen, and how they happen. This book doesn't just present a series of cool sounds and techniques, it also attempts to unpack and dissect important details, show alternatives, illustrate common misunderstandings, and contextualize these techniques within a broader creative framework.

This book, and other teaching materials I've published, exist with a central goal of simplifying and accelerating the learning process for anyone who's recently discovered SuperCollider and feels a similar spark. Having taught music technology in higher education for several years, I've encountered several types of students for whom I think this book would be valuable. As the title suggests, it's a structured tutorial and reference guide for anyone wanting to create electronic music or experimental sound art with SuperCollider, and assumes readers fall somewhere on the continuum between musician and computer programmer.

Maybe you're a seasoned musician with a budding interest in coding, or maybe you're a software developer who creates electronic music as a side hustle. Whatever the case, if you have some familiarity with music and code—even just a little of each—you should be able to use this book to pick up the basics and get some new creative projects off the ground. Keep in mind, however, that this book isn't a formal substitute for an introduction to programming, digital audio principles, or music theory. These are very broad topics, beyond the scope of this publication. They are selectively discussed where relevant, but the overall focus of this book remains on using SuperCollider to develop creative audio projects.

This book divides into three parts: Part I covers the fundamentals of using the language, expressing ideas through code, writing basic programs, and producing simple sounds. Part II focuses on common practice techniques, including synthesis, sampling, sequencing, signal processing, external controllers, and graphical user interface design. In short, this second part details the “ingredients” that make up a typical project. Part III brings it all together and offers concrete strategies for composing, rehearsing, and performing large-scale projects.

It's hard to overstate the value of hands-on learning, especially for a sound-oriented coding language. To this end, each chapter includes numerous in-line SuperCollider code examples, and each chapter also pairs with one or more “Companion Code” files, which explore chapter topics in significantly greater depth. All of the Code Examples and Companion Codes are hosted on a companion website and referenced in the text with the following icon:



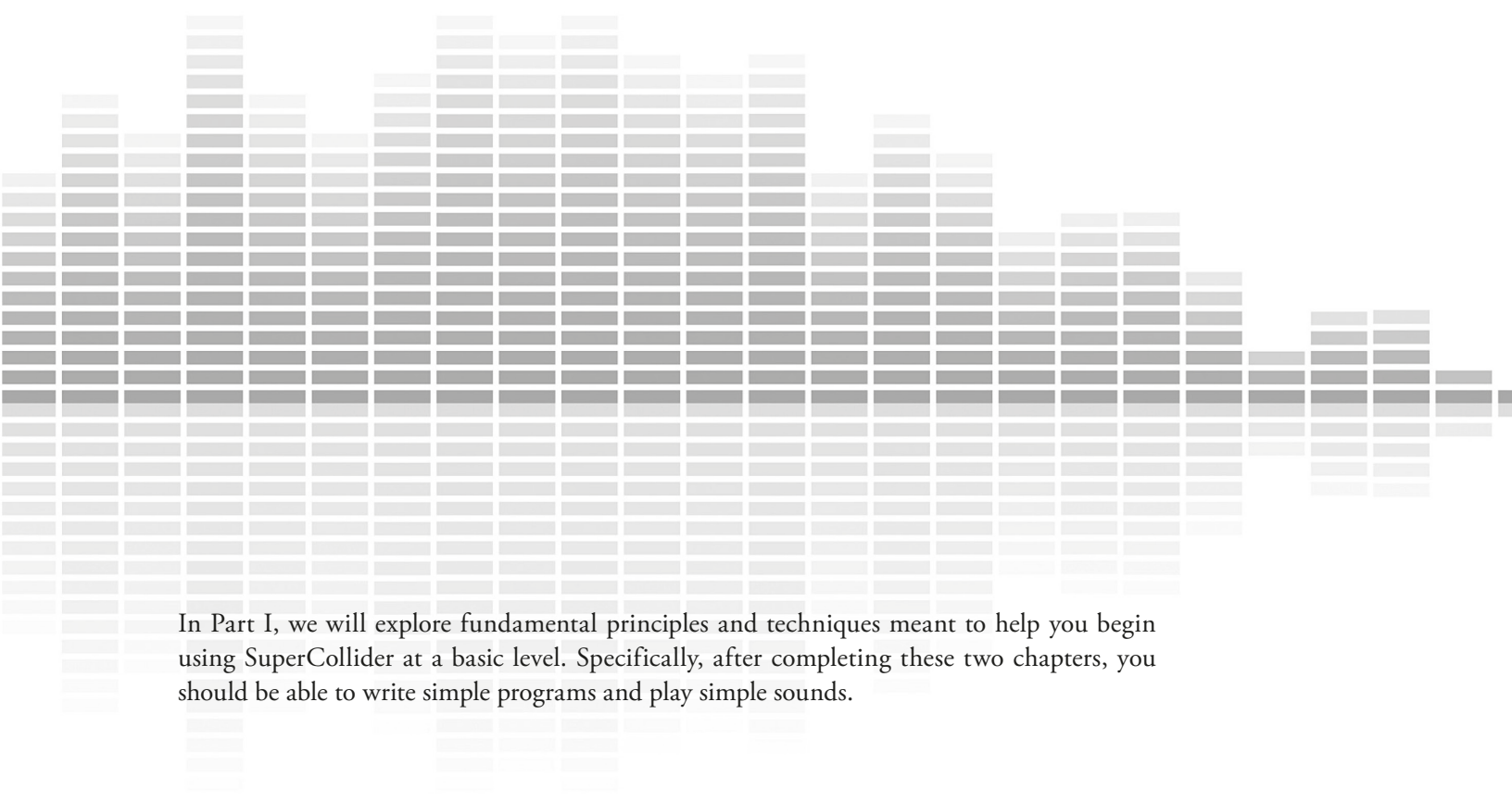
Use these resources! As you read, you'll (hopefully) get the urge to experiment. When you do, download these files, study their contents, and play around with them. These files exist to provide immediate, seamless access, avoiding the need to manually copy code from the book. Learning SuperCollider is a bit like learning a foreign language; reading a textbook is certainly a good idea, but if you want to become fluent, there's no proper substitute for immersing yourself in the language. You'll make some mistakes, but you'll also make important discoveries. It's all part of the learning process!

Before you begin reading, download the SuperCollider software, which (at the time of writing) is available at supercollider.github.io. You may also want to visit/bookmark the SuperCollider project page on GitHub (github.com/supercollider/supercollider), which includes a wiki, links to tutorials, and many other resources. I recommend working with a quality pair of headphones or loudspeakers, although built-in laptop speakers will work in a pinch. A good quality microphone and audio interface can be useful for exploring topics in Chapter 6, but there's plenty to keep you busy if you don't have access to these things, and a built-in laptop microphone will suffice for basic exploration. A standard MIDI keyboard controller will also be helpful (though not essential) for exploring some of the code examples in Chapters 7–8.

Finally, as you work through this book, keep an open mind and be patient with yourself. There's a lot to learn and it won't happen overnight, but with practice and dedication, you'll find that an amazing new world of sound will begin to reveal itself, which can be an incredibly exciting and liberating experience. Onward!

PART I

FUNDAMENTALS



In Part I, we will explore fundamental principles and techniques meant to help you begin using SuperCollider at a basic level. Specifically, after completing these two chapters, you should be able to write simple programs and play simple sounds.

CHAPTER 1

CORE PROGRAMMING CONCEPTS

1.1 Overview

This chapter covers the essentials of programming in SuperCollider, such as navigating the environment, getting acquainted with basic terminology, understanding common data types, making simple operations, and writing/running simple programs. The goal is to become familiar with things you'll encounter on an everyday basis and develop a foundational understanding that will support you through the rest of this book and as you embark on your own SC journey.

Keyboard shortcuts play an important role in SC, as they provide quick access to common actions. Throughout this book, the signifier [cmd] refers to the command key on macOS systems, and the control key on Windows and Linux systems.

1.2 A Tour of the Environment

Before we start dealing with code, a good first step is to understand what we see when opening SC for the first time, as depicted in Figure 1.1.

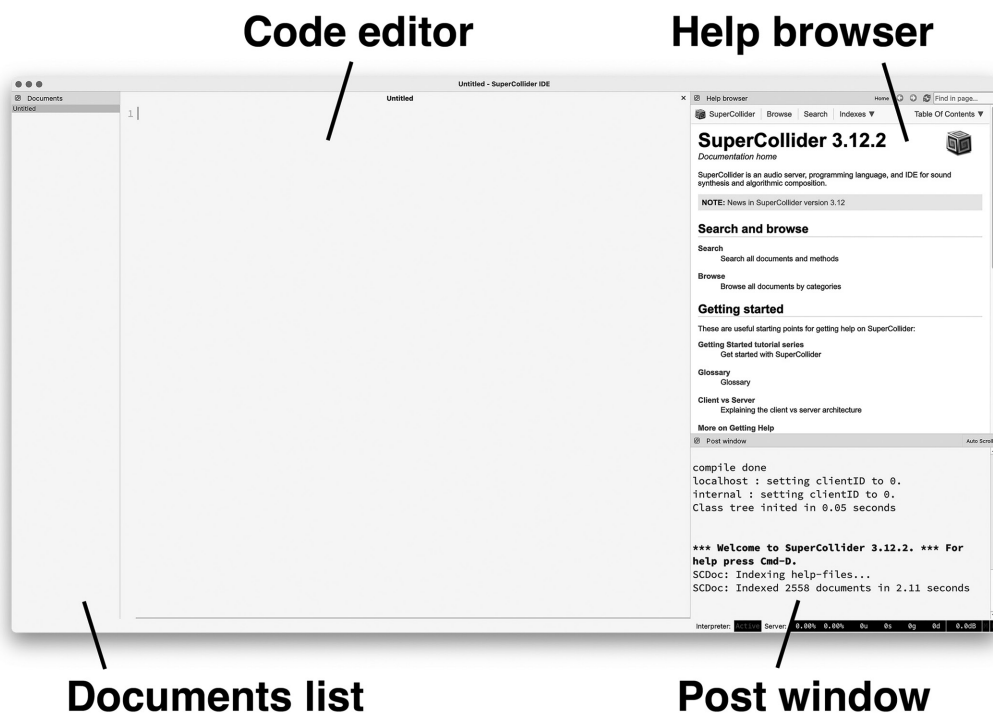


FIGURE 1.1 The SuperCollider environment.

The centerpiece of the SC environment is the code editor, a large blank area that serves as your workspace for creating, editing, and executing code. Like a web browser, multiple files can be open simultaneously, selectable using the tab system that appears above the editor. Code files can be saved and loaded as they would be in any text editing or word processing software. SC code files use the file extension “scd” (short for SuperCollider document). The code editor is surrounded by three docklets: the post window, the help browser, and the documents list. The post window is where SC displays information for the user. Most often, this is the result of evaluated code, but may also include error messages and warnings. The help browser provides access to documentation files that detail how SC works. The documents list displays names of files that are currently open, offering a handy navigation option if many documents are open at once. Altogether, these and other features constitute the SC Integrated Development Environment, commonly known as the IDE. The IDE serves as a front-end interface for the user, meant to enhance workflow by providing things like keyboard shortcuts, text colorization, auto-completion, and more. The preferences panel, which is shown in Figure 1.2, is where some of these customizations can be made. The preferences panel is accessible in the “SuperCollider” drop-down menu on macOS, and the “Edit” drop-down menu on Windows/Linux.

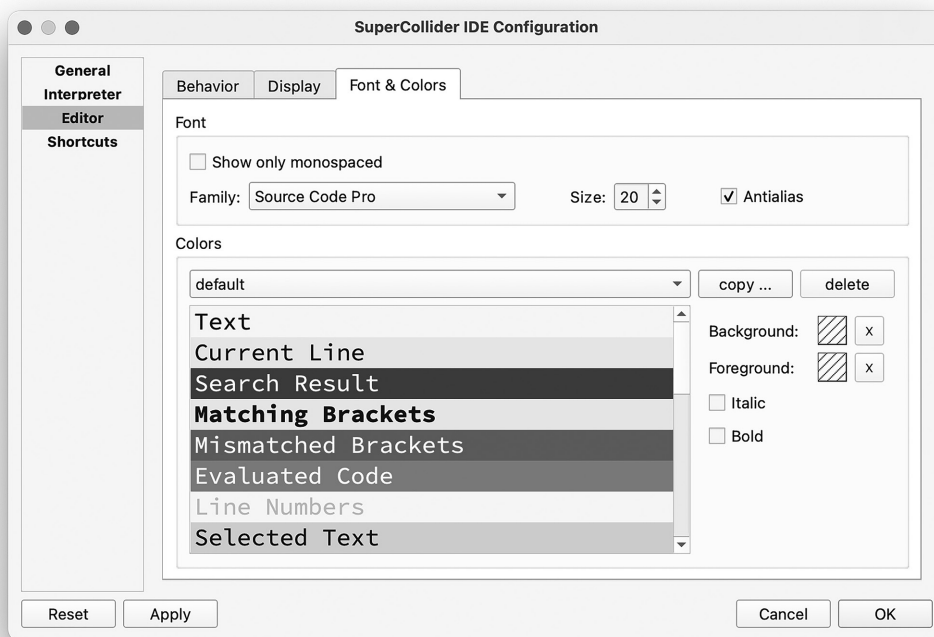


FIGURE 1.2 The SuperCollider preferences panel.

The IDE is one of three components that make up the SC environment. In addition, there is the SC language (often called “the language” or “the client”) and the SC audio server (often called “the server”). The technical names for these components are *sclang* and *scsynth*, respectively, and despite being integral parts of the environment, they remain largely invisible to the user.

The language is home to a library of objects, which are the basic building blocks of any program. It also houses the interpreter, a core component that interprets and executes

code, translating it into action. The interpreter automatically starts when the IDE is launched, and a small status box in the lower-right corner of the IDE displays whether the interpreter is active. SC is an interpreted language, rather than a compiled language. In simple terms, this means that code can be executed selectively and interactively, on a line-by-line basis, rather than having to write the entirety of a program before running it. This is especially useful because SC is regularly used for real-time creative audio applications. In some musical scenarios (improvisation, for example), we won't know all the details in advance, so it's convenient to be able to interact with code "in the moment." This dynamism is featured in examples throughout this book (many of which are broken into separate chunks of code) and is on full display in the last chapter of this book, which discusses live coding.

The audio server is the program that handles calculation, generation, and processing of audio signals. Colloquially, it's the "engine" that powers SC's sound capabilities. It cannot be overstated that the server is fully detached and fundamentally independent from the language and IDE. The language and the server communicate over a network using a client-server model. For example, to play some audio, we first evaluate some code in the language. The language, acting as a client on the network, sends a request to the server, which responds by producing sound. In a sense, this setup is like checking email on a personal computer. An email client gives the impression that your messages are on your own machine, but in reality, you're a client user on a network, sending requests to download information stored on a remote server.

Often, the language and server are running on the same computer, and this "network" is more of an abstract concept than a true network of separate, interconnected devices. However, the client-server design facilitates options involving separate, networked machines. It's possible for an instance of the language on one computer to communicate with an audio server running on a different computer. It's equally possible for multiple clients to communicate with a single server (a common configuration for laptop ensembles), or for one language-side user to communicate with multiple servers. Messages passed between the language and server rely on the Open Sound Control (OSC) protocol, therefore the server can be controlled by any OSC-compliant software.¹ Most modern programming languages and audiovisual environments are OSC-compliant, such as Java, Python, Max/MSP, and Processing, to name a few. Even some digital audio workstations can send and receive OSC. However, using an alternative client doesn't typically provide substantial benefits over the SC language, due to its high-level objects that encapsulate and automatically generate OSC messages. In addition, the client-server divide improves stability; if the client application crashes, the server can play on without interruption while the client is rebooted.

Unlike the interpreter, the server does not automatically start when the IDE is launched and must be manually booted before we can start working with sound, which is the focus of the next chapter. For now, our focus is on the language and the IDE.

1.3 An Object-Oriented View of the World

The SC language is an object-oriented language. Though it's helpful to understand some core principles of object-oriented programming (OOP), a deep dive is not necessary from a musical perspective. This section conveys some essentials of OOP, relying primarily on analogies from the real world, avoiding a more theoretical approach.

1.3.1 OBJECTS AND CLASSES

Our world is full of *objects*, such as plants, animals, buildings, and vehicles. These objects exist within a hierarchy: some objects, like houses, supermarkets, and stadiums, are all members of the same category, or *class* of objects, called buildings. Applying some programming terminology to this example, buildings would be considered the parent class (or superclass) of houses, supermarkets, stadiums, and many others. Similarly, houses, supermarkets, and stadiums are each a child class (or subclass) of buildings. This hierarchy might extend further in either direction: we might have several subclasses of houses (cottages, mansions, etc.), while buildings, vehicles, plants, and animals might be subclasses of an even more all-encompassing superclass named “things.” This organizational structure is tree-like, with a few large, broad branches near one end, and many smaller and more specific branches toward the other end.

Like the real world, the SC language is full of objects that exist within a similar class hierarchy, called the *class tree*. Classes of objects in SC include things like Integers, Floats, Strings, Arrays, Functions, Buttons, Sliders, and much more. When represented with code, a class name always begins with a capital letter, which is how SC distinguishes them. If some of these terms are foreign to you, don’t worry. We’ll introduce them properly later on.

1.3.2 METHODS AND INHERITANCE

Certain types of interactions make sense for certain types of objects. For example, it’s reasonable to ask whether an animal is a mammal, but nonsense when applied to a vehicle. On the other hand, some types of queries make sense for all objects, like asking whether something is alive. The point is that a specific type of object “knows” how to respond to certain types of inquiries, to which it responds in a meaningful way. Such queries are called *methods* or *messages*.² For example, an integer knows how to respond to being “squared,” (multiplied by itself). To a graphical slider, this method is meaningless. Further still, there are some methods that have meaning for both integers and sliders, such as one that asks, “are you an integer?”

If a method is defined for a class, all its subclasses automatically inherit that method and respond to it in a similar manner. It’s also possible for the same method to produce different results when applied to different objects (this ability is called *polymorphism*). Consider the verb “to file.” Semantically, what this means depends on the object being filed. To file a piece of paper means to organize it in a filing cabinet. To file a piece of metal, on the other hand, means to smooth it by grinding it down. Many examples of polymorphism exist within SC. The “play” method, for instance, is defined for many types of objects, but produces various results that may or may not involve sound production.

1.3.3 CLASSES VS. INSTANCES

There is an important distinction between a tangible object, and the abstract idea of that object. To illustrate, consider a house, compared to a blueprint for a house. The blueprint represents a house and provides a set of instructions for creating one. But it’s not the same thing as the house itself. You can’t live in a blueprint!

In this example, the blueprint functions similarly to the *class* of houses, while a physical house would be considered an *instance* of that class. You could, for example, use one blueprint to construct many houses. It might even be possible to construct several different houses using the same blueprint, by making a few tweaks to the instructions before each build. Generally, a class can conjure into existence a tangible version of what it represents, but it’s usually these instances that we are interested in working with.

1.4 Writing, Understanding, and Evaluating Code

As we begin translating these abstract concepts into concrete code, bear in mind that writing computer code requires precision. Even the most seemingly minor details, like capitalization and spelling, are important. The interpreter will never try to “guess” what you mean if your code is vague or nonsensical. On the contrary, it will attempt to do exactly what you instruct and report errors if it can’t comply.

Type the following into the code editor, exactly as it appears:

```
4.squared;
```

Then, making sure your mouse cursor is placed somewhere on this line, press [shift]+[return], which tells the interpreter to execute the current line. You’ll see a brief flash of color over the code, and the number 16 will appear in the post window. If this is your first time using SC, congratulations! You’ve just run your first program. Let’s take a step back and dissect what just happened.

1.4.1 A SINGLE LINE OF CODE, DECONSTRUCTED

The expression **4.squared** contains a member of the integer class and the “squared” method, which is defined for integers. We say that four is the *receiver* of the method. The period, in this context, is the symbol that applies the method to the receiver. This “receiver-dot-method” construction is commonly used to perform operations with objects, but there is a syntax alternative that involves placing the receiver in an enclosure of parentheses, immediately following the method:

```
squared(4);
```

Why choose one style over the other? This is ultimately dictated by personal preference and usually governed by readability. For example, an English speaker would probably choose **4.squared** instead of **squared(4)**, because it mimics how the phrase would be spoken.

The parentheses used to contain the receiver are one type of *enclosure* in SC. Others include [square brackets], {curly braces}, “double quotes,” and ‘single quotes.’ Each type of enclosure has its own significance, and some have multiple uses. We’ll address enclosures in more detail as they arise, but for now it’s important to recognize that an enclosure always involves a pair of symbols: one to begin the enclosure, and another to end it. Sometimes, an enclosure contains only a few characters, while others might contain thousands of lines. If an opening bracket is missing its partner, your code won’t run properly, and you’ll likely see an error message.

The semicolon is the statement terminator. This symbol tells the interpreter where one expression ends and, possibly, where another begins. Semicolons are the code equivalent of periods and question marks in a novel, which help our brain parse the writing into discrete sentences. In SC, every code statement should always end with a semicolon. Omitting a semicolon is fine when evaluating only a single expression, but omitting a semicolon in a multi-line situation will usually produce an error.

In the case of either **4.squared** or **squared(4)**, the result is the same. On evaluation, the interpreter *returns a value* of 16. Returning a value is not the same thing as printing it in the post window. In fact, the interpreter always posts the value of the last evaluated line, which

makes this behavior more of a byproduct of code evaluation than anything else. Returning a value is a deeper, more fundamental concept, which means the interpreter has taken your code and digested it, and is passing the result back to you. We can think of the expression **4.squared** or **squared(4)** as being an equivalent representation of the number 16. As such, we can treat the entire expression as a new receiver and chain additional methods to apply additional operations. For instance, Code Example 1.1 shows how we can take the reciprocal (i.e., one divided by the receiver) of the square of a number, demonstrated using both syntax styles:

CODE EXAMPLE 1.1: COMBINING MULTIPLE METHODS INTO A SINGLE STATEMENT, USING TWO DIFFERENT SYNTAX STYLES.

```
4.squared.reciprocal;  
  
reciprocal(squared(4));
```

Methods chained using the “receiver-dot-method” style are applied from left to right. When using the “method(receiver)” syntax, the flow progresses outward from the innermost set of parentheses.

Unlike **squared** or **reciprocal**, some methods require additional information to return a value. For instance, the **pow** method raises its receiver to the power of some exponent, which the user must provide. Code Example 1.2 shows an expression that returns three to the power of four, using both syntax styles. In this context, we say that four is an *argument* of the **pow** method. Arguments are a far-reaching concept that we’ll explore later in this chapter. For now, think of an argument as an input value needed for some process to work, such as a method call. Attempts to use **pow** without providing the exponent will produce an error. Note that when multiple items appear in an argument enclosure, they must be separated with commas.

CODE EXAMPLE 1.2: A CODE EXPRESSION INVOLVING A RECEIVER, METHOD, AND ARGUMENT, USING TWO DIFFERENT SYNTAX STYLES.

```
3.pow(4);  
  
pow(3, 4);
```

1.4.2 MULTIPLE LINES OF CODE, DECONSTRUCTED

Suppose we continued chaining additional methods to one of the expressions in Code Example 1.1. After several dozen methods, the line would become quite long. It might run

over onto a new line and would also become increasingly difficult to read and understand. For this reason, we frequently break up a series of operations into multiple statements, written on separate lines. To do this correctly, we need some way to “capture” a value, so that it can be referenced in subsequent steps.

A *variable* is a named container that can hold any type of object. One way to create a variable is to declare it using a **var** statement. A variable name must begin with a lowercase letter. Following this letter, the name can include uppercase/lowercase letters, numbers, and/or underscores. Table 1.1 contains examples of valid and invalid variable names.

TABLE 1.1 Examples of valid and invalid variable names.

Valid	Invalid
num	Num
myValue	my#\$\$&Value
sample_04	sample-04

Variable names should be short, but meaningful, striking a balance between brevity and readability. Once a variable is declared, an object can be assigned to a variable using the equals symbol. A typical sequence involves declaring a variable, assigning a value, and then repeatedly modifying the value and assigning each new result to the same variable, overwriting the older assignment in the process. This initially looks strange from a “mathematical” perspective, but the equals symbol doesn’t mean numerical equality in this case; instead, it denotes a storage action. In Code Example 1.3, the expression **num = num.squared** could be translated into English as the following command: “Square the value currently stored in the variable named ‘num,’ and store the result in the same variable, overwriting the old value.”

To evaluate a multi-line block, we need to learn a new technique and a new keyboard shortcut. The code in Code Example 1.3 has been wrapped in an enclosure of parentheses, each on its own line above and below. Here, we’re already seeing that a parenthetical enclosure serves multiple purposes. In this case, it delineates a multi-line block: a modular unit that can be passed to the interpreter with a single keystroke. Instead of pressing [shift]+[return], place the cursor anywhere inside the enclosure and press [cmd]+[return].

CODE EXAMPLE 1.3: A MULTI-LINE BLOCK OF CODE.

```
(
var num;
num = 4;
num = num.squared;
num = num.reciprocal;
)
```

TIP.RAND(); EVALUATING CODE

Using the correct technique and keystroke for evaluating code can be a common point of confusion for new users. Even if your code is written correctly, misevaluation will produce errors that suggest otherwise, because it causes the interpreter to “see” your code incorrectly. Remember that there are two keyboard shortcuts for evaluating code: [shift]+[return] and [cmd]+[return]. Their behavior depends on the following conditions:

- If any code is highlighted:
 - either [cmd]+[return] or [shift]+[return] will evaluate the highlighted code.
- If no code is selected, and the cursor is inside a multi-line block enclosure:
 - [cmd]+[return] will evaluate the entire block.
 - [shift]+[return] will evaluate the line on which the cursor is placed.
- If no code is selected, and the cursor is not inside a multi-line block enclosure:
 - either [cmd]+[return] or [shift]+[return] will evaluate the line on which the cursor is placed.

Also note that in this context, the presence of return characters (rather than semicolons) determines what constitutes a “line” of code. For example, despite containing multiple statements, the following example is considered a single line, so either [cmd]+[return] or [shift]+[return] will evaluate it:

```
var num; num = 4; num = num.squared; num = num.reciprocal;
```

Conversely, the following example is considered to occupy multiple lines, despite containing only one statement. Thus, [shift]+[return] will not work unless the entire example is highlighted, and [cmd]+[return] will not work unless contained in a parenthetical enclosure.

```
4
.squared;
```

Variables declared using a **var** statement are *local variables*; they are local to the evaluated code of which they are a part. This means that once evaluation is complete, variables created in this way will no longer exist. If you later attempt to evaluate **num** by itself, you’ll find that it not only has lost its value assignment, but also produces an error indicating that the variable is undefined. Local variables are transient. They are useful for context-specific cases where there’s no need to retain the variable beyond its initial scope.

If we need several local variables, they can be combined into a single declaration. They can also be given value assignments during declaration. Code Example 1.4 declares three variables and provides initial assignments to the first two. We square the first variable, take the reciprocal of the second, and return the sum, which is 49.2.

CODE EXAMPLE 1.4: A MULTI-LINE CODE BLOCK THAT DECLARES MULTIPLE LOCAL VARIABLES AND PROVIDES DEFAULT ASSIGNMENTS.

```
(  
var thingA = 7, thingB = 5, result;  
thingA = thingA.squared;  
thingB = thingB.reciprocal;  
result = thingA + thingB;  
)
```

TIP.RAND(); NEGATIVE NUMBERS AS DEFAULT VARIABLE ASSIGNMENTS

If a variable is given a negative default value during declaration, there's a potential pitfall. The negative value must either be in parentheses or have a space between it and the preceding equals symbol. If the equals symbol and the minus symbol are directly adjacent, SC will mistakenly interpret the pair of symbols as an undefined operation. This same rule applies to a declaration of arguments, introduced in the next section. Of the following three expressions, the first two are valid, but the third will fail.

```
var num = -2;
```

```
var num =(-2);
```

```
var num ==-2;
```

What if we want to retain a variable, to be used again in the future as part of a separate code evaluation? We can use an *environment variable*, created by preceding the variable name with a tilde character (~). Alternatively, we can use one of the twenty-six lowercase alphabetic characters, which are reserved as *interpreter variables*. Both environment and interpreter variables can be used without a local declaration, and both behave with global scope, that is, they will retain their value (even across multiple code documents) as long as the interpreter remains active. Interpreter variables are of limited use, since there are only twenty-six of them and they cannot have longer, more meaningful names. But they are convenient for bypassing declaration when sketching out an idea or quickly testing something. Environment variables

are generally more useful because we can customize their names. The two blocks of code in Code Example 1.5 are each equivalent to the code in Code Example 1.3, but in either case, the variable will retain its assignment after evaluation is complete.

CODE EXAMPLE 1.5: USAGE OF INTERPRETER AND ENVIRONMENT VARIABLES (RESPECTIVELY).

```
(  
  n = 4;  
  n = n.squared;  
  n = n.reciprocal;  
)  
  
(  
  ~num = 4;  
  ~num = ~num.squared;  
  ~num = ~num.reciprocal;  
)
```

1.4.3 POSTING A VALUE

The **postln** method has the effect of printing its receiver to the post window, followed by a new line. This method simply returns its receiver, so it is “neutral” in the sense that it does not modify the object to which it applies. This method is useful for visualizing values partway through a multi-line block, perhaps to check for correctness. On evaluating the code in Code Example 1.6, we will see the value 16 appear in the post window (the result of **postln**), followed by 0.0625 (the result of the interpreter posting the result of the last evaluated line).

CODE EXAMPLE 1.6: USE OF THE `postln` METHOD.

```
(  
  ~num = 4;  
  ~num = ~num.squared.postln;  
  ~num = ~num.reciprocal;  
)
```

1.4.4 COMMENTS

A comment is text that the interpreter ignores. Comments are typically included to provide some information for a human reader. The imagined reader might be someone else, like a friend or collaborator, but it also might be you! Leaving notes for yourself is a good way to jog your memory if you take a long break from a project and don’t quite remember where you left off.

There are two ways to designate text as a comment. For short comments, preceding text with two forward slashes will “comment out” the text until the next return character. To create larger comments that include multiple lines of text, we enclose the text in another type of enclosure, beginning with a forward slash–asterisk, and ending with an asterisk–forward slash. These styles are depicted in Code Example 1.7.

CODE EXAMPLE 1.7: USAGE OF TWO DIFFERENT COMMENT STYLES.

```
(
/*
this is a multi-line comment, which might
be used at the top of your code in order
to explain some features of your program.
*/
var num = 4; // a single-line comment: declare a variable
num = num.squared;
num = num.reciprocal;
)
```

1.4.5 WHITESPACE

Whitespace refers to the use of “empty space” characters, like spaces and new lines. Generally, SC is indifferent to whitespace. For example, both of the following statements are considered syntactically valid and will produce the same result:

```
4.squared+2;
```

```
4 . squared + 2;
```

The use of whitespace is ultimately a matter of taste, but a consensus among programmers seems to be that there is a balance to be struck. Using too little whitespace cramps your code and doesn’t provide enough room for it to “breathe.” Too much whitespace, and code spreads out to the point of being similarly difficult to read. The whitespace choices made throughout this book are based on personal preference and years of studying other programmers’ code.

1.5 Getting Help

The IDE includes resources for getting help on a number of topics. This section focuses on accessing and navigating these resources.

1.5.1 THE HELP BROWSER

The help browser is the primary resource for getting help within the IDE. It provides access to all of SC’s documentation, which includes overviews, reference files, guides, a few tutorials,

and individual help files for every object.³ It includes a search feature, as well as browse option for exploring the documentation by category. It's a good idea to spend some time browsing to get a sense of what's there. Keep in mind that the documentation is not structured as a comprehensive, logically sequenced tutorial (if it were, there'd be less need for books such as this); rather, it tends to be more useful as a quick reference for checking some specific detail.

There are two keystrokes for quickly accessing a page in the help browser: `[cmd]+[d]` will perform a search for the class or method indicated by the current location of the mouse cursor, and `[shift]+[cmd]+[d]` invokes a pop-up search field for querying a specific term. If there is an exact class match for your query (capitalization matters), that class's help file will appear. If not, the browser will display a list of results that it thinks are good matches. Try it yourself! Type "Integer" into the editor, and press `[cmd]+[d]` to bring up the associated help file.

The structure of a class help file appears in Figure 1.3. In general, these files each begin with the class name, along with its superclass lineage, a summary of the class, and a handful of related classes and topics. Following this, the help file includes a detailed description, class methods, instance methods, and often some examples that can be run directly in the browser and/or copied into the text editor.

Help browser
Home
Find in page...

SuperCollider
Browse
Search
Indexes
Table Of Contents

Classes | Core > Kernel

Routine

: Thread : Stream : AbstractFunction : Object

Functions that can return in the middle and then resume where they left off

Source: Thread.sc
Subclasses: FuncStreamAsRoutine

See also: Stream

Description

A Routine runs a Function and allows it to be suspended in the middle and be resumed again where it left off. This functionality is supported by the Routine's superclass Thread. Effectively, Routines can be used to implement co-routines as found in Scheme and some other languages.

Class Methods

Routine.new(func, stackSize: 512)

From superclass: Thread

Creates an instance of Routine, passing it the Function with code to run.

Arguments:

- func** A Function with code for the Thread to run.
- stackSize** Call stack size (an Integer).

Discussion:

```
a = Routine.new({ 1.yield; 2.yield; });
a.next.postln;
a.next.postln;
a.next.postln;
```

Instance Methods

.next(inval)

This method performs differently according to the Routine's state:

- Starts the Routine, if it has not been started yet or it has been reset; i.e runs its Function from the beginning, passing on the `inval` argument.
- Resumes the Routine, if it has been suspended (it has yielded); i.e. resumes its Function from the point where `yield` was called on an Object, passing the `inval` argument as the return value of `yield`.
- Does nothing if the Routine has stopped (because its Function has returned, or -stop has been called).

Examples

```
(
  var r, outval;
  r = Routine.new({ arg inval;
    ("->inval was " ++ inval).postln;
    inval = 1.yield;
    ("->inval was " ++ inval).postln;
    inval = 2.yield;
    ("->inval was " ++ inval).postln;
    inval = 99.yield;
  });
);
```

FIGURE 1.3 The structure of a class help file. Vertical ellipses represent material omitted for clarity and conciseness.

1.5.2 SEARCHING FOR IMPLEMENTATIONS

The help browser is the primary resource for new users, but as you get more comfortable with SC, you may want to look directly at the source code for a class or method. Additionally, because SC is an open-source project in active development, there are cases where the content of a help file may not be completely consistent with the behavior of the object it describes. The source code, on the other hand, lets us see exactly how something works.

There are two keystrokes for looking up implementations of class or method: `[cmd]+[i]` will perform a search for all the source code implementations of the class or method indicated by the current location of the mouse cursor, and `[shift]+[cmd]+[i]` will bring up a search bar. When you enter your search term, a list of source code files in which that term appears will populate a list (see Figure 1.4). Selecting one will open the source file and automatically scroll to the relevant spot. Just be sure not to edit anything!

To try this yourself, press `[shift]+[cmd]+[i]`, type “play” in the search bar, and press enter to see a list of classes that implement this method. Click one of these items and press enter to open the source code for that class. If the source code looks totally incomprehensible to you, don’t worry! Being able to read these files isn’t essential, but as you get more comfortable with SC, you may find yourself digging around in these files more often.

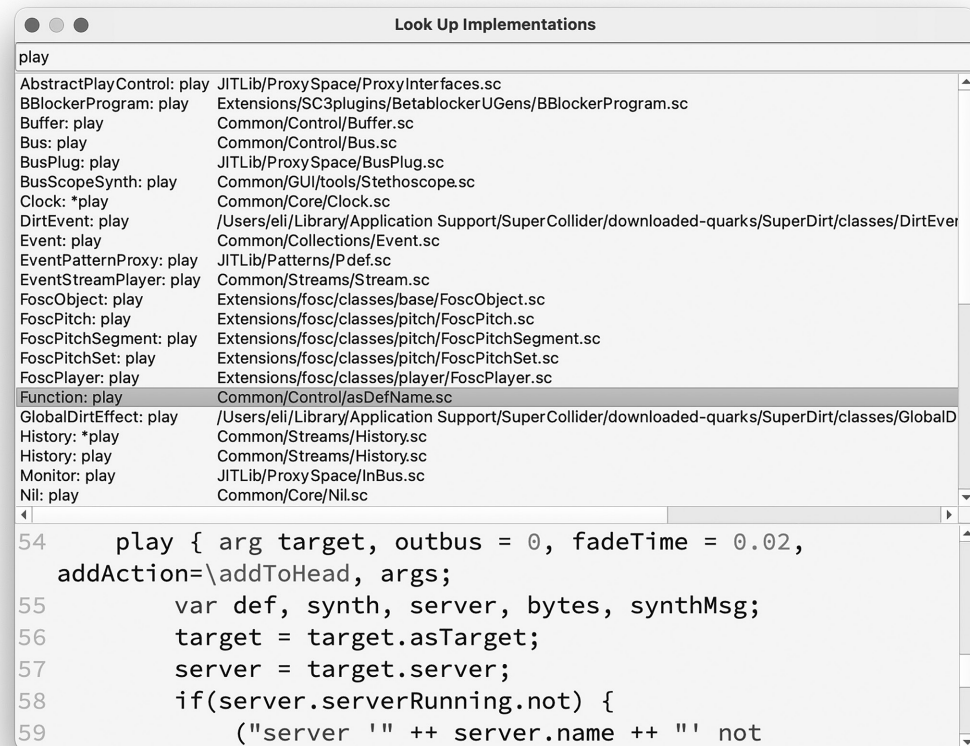


FIGURE 1.4 The search window for looking up implementations of methods.

1.5.3 THE CLASS BROWSER

The class browser is a graphical tool for browsing the class tree. The **browse** method can be applied to any class name (e.g. **Array.browse**), which invokes the browser and displays information for that class. The class browser, shown in Figure 1.5, displays a class's superclass, subclasses, methods, and arguments. Buttons near the top of the browser can be used for navigating backward and forward (like a web browser), as well as navigating to a class's superclass, or bringing up source code files. You can also navigate to a subclass by double-clicking it in the list in the bottom-left corner.

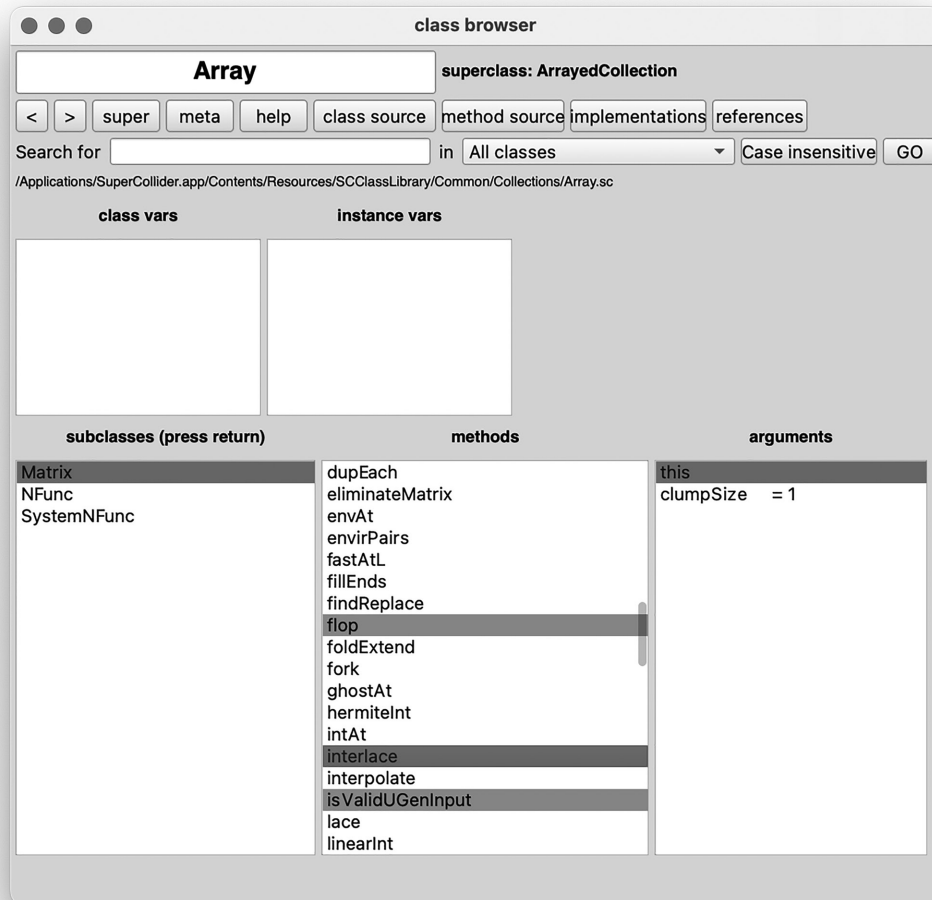


FIGURE 1.5 The class browser.

1.6 A Tour of Classes and Methods

With a basic understanding of writing and evaluating code behind us, we're ready to take a more detailed look at some commonly used classes and methods, to get a better sense of the operations we're likely to regularly encounter.

1.6.1 INTEGERS AND FLOATS

The **Integer** and **Float** classes represent numerical values. We've already encountered integers, which are whole numbers that can be positive, negative, or zero. A float, on the other hand, is a number that includes a decimal point, like 9.02 and -315.67. Even the number 1.0 is a float, despite being quantitatively equal to one. Some programming languages are quite particular about the distinction between integers and floats, balking at certain operations that attempt to combine them, but SC is flexible, and will typically convert automatically as needed. Integers and floats are closely related on the class tree and therefore share many methods, but there are a few methods which apply to one and not the other.

We'll begin our tour by introducing the **class** method, which returns the class of its receiver. This method is not exclusive to integers and floats; in fact, every object knows how to respond to it! However, it's relevant here in showing the classes to which different numbers belong.

```
4.class; // -> Integer
```

```
4.0.class; // -> Float
```

There are many methods that perform mathematical operations with numbers. Those that perform an operation involving two values are called *binary operators*, while those that perform an operation on a single receiver, like taking the square root of a number, are called *unary operators*. Most operations exist as method calls (e.g., **squared**), but some, like addition and multiplication, are so commonly used that they have a direct symbolic representation (imagine if multiplication required typing **12.multipliedBy(3)** instead of **12 * 3**). Descriptive lists of commonly used operators appear in Tables 1.2 and 1.3. A more complete list can be found in a SC overview file titled "Operators."

TABLE 1.2 List of common unary operators.

Method Usage	Description
abs(x);	Absolute value. Non-negative value of x , i.e., distance from zero.
ceil(x);	Round up to the nearest whole number.
floor(x);	Round down to the nearest whole number.
neg(x);	Negation. Positive numbers become negative and vice-versa.
reciprocal(x);	Return 1 divided by x .
sqrt(x);	Return the square root of x .
squared(x);	Return x raised to the power of 2.

TABLE 1.3 List of common binary operators.

Name	Method Usage	Symbolic Usage	Description
Addition	N/A	x + y;	
Subtraction	N/A	x - y;	